

Projet de Programmation

Recherche de chemins optimaux sous contrainte de fatigue

Evan BEGRICHE et Gaspard PICHON

Année scolaire 2025-2026

ENSAE Paris

Résumé

Ce projet traite d'un problème de plus court chemin où le coût de traversée d'une route dépend de la fatigue accumulée. Ce rapport détaille et justifie nos choix algorithmiques pour contourner l'explosion de la complexité (graphe implicite, élagage de Pareto, heuristiques A*). Nous appliquons ensuite cette base à la résolution d'un problème de planification de missions multiples (ordre imposé et libre), avant de présenter et d'analyser nos résultats expérimentaux.

Table des matières

1	Modélisation et Justification de la Complexité	2
1.1	Le problème des coûts dynamiques	2
1.2	Complexité du Graphe Étendu (L'approche naïve)	2
2	Choix Algorithmiques et Justifications	2
2.1	Choix 1 : Le Graphe Implicite (Évaluation Paresseuse)	2
2.2	Choix 2 : L'Élagage de Pareto (Pruning)	2
2.3	Choix 3 : A* et Conception des Heuristiques	2
3	Choix Algorithmiques pour les Missions Multiples	3
3.1	Missions avec Ordre Imposé	3
3.2	Missions avec Ordre Libre	3
3.2.1	Gestion des collisions dans la file de priorité (L'état "ZZZ_FIN")	3
4	Présentation et Analyse des Résultats	3
4.1	Comparaison des algorithmes (Dijkstra vs A*)	3
4.2	Résultats de l'Extension (Missions Multiples)	4
5	Conclusion	5

1 Modélisation et Justification de la Complexité

1.1 Le problème des coûts dynamiques

Dans notre réseau $G = (V, E)$, chaque route e possède une longueur $l(e)$ et génère une fatigue $f(e)$. Le coût pour l'emprunter, si l'on a déjà accumulé une fatigue F , est :

$$c(e, F) = l(e) \times (1 + F)$$

Conséquence théorique : L'optimalité des sous-chemins n'est plus garantie. Un détour qui allonge la distance mais réduit la fatigue accumulée peut s'avérer optimal sur le long terme. Les algorithmes classiques (comme Dijkstra) appliqués tels quels sur la carte géographique donneront des résultats faux.

1.2 Complexité du Graphe Étendu (L'approche naïve)

Pour utiliser Dijkstra, il faut changer la définition d'un nœud. Un nœud devient un état (v, F) : la ville v et la fatigue F . Si $F_{\max}$ est la fatigue maximale atteignable, la taille de notre nouveau graphe devient :

- Nombre de sommets : $\mathcal{O}(|V| \times F_{\max})$
- Nombre d'arêtes : $\mathcal{O}(|E| \times F_{\max})$

Justification de l'échec : La complexité spatiale (mémoire) explose. Calculer et stocker ce "Graphe Étendu" à l'avance sature la RAM sur les instances moyennes et grandes, provoquant une `MemoryError`. Il fallait donc changer de stratégie algorithmique.

2 Choix Algorithmiques et Justifications

Pour résoudre les très grandes instances tout en respectant les contraintes de mémoire et de temps, nous avons fait trois choix algorithmiques majeurs.

2.1 Choix 1 : Le Graphe Implicite (Évaluation Paresseuse)

Le choix : Au lieu de créer tous les états (v, F) en mémoire au début du programme, nous avons implémenté une classe `GraphImplicit` qui génère les voisins d'un état uniquement lorsque l'algorithme visite cet état.

La justification : La complexité spatiale passe de $\mathcal{O}(|V| \times F_{\max})$ au nombre exact d'états réellement explorés par l'algorithme. La topologie complète n'est jamais stockée. C'est ce choix qui permet au programme de ne plus crasher sur les grands réseaux.

2.2 Choix 2 : L'Élagage de Pareto (Pruning)

Le choix : Nous avons ajouté un dictionnaire pour mémoriser la fatigue minimale avec laquelle nous avons atteint chaque ville v . Si l'algorithme découvre un nouveau chemin menant à v avec une fatigue F_2 supérieure ou égale à la meilleure fatigue connue F_1 , ce chemin est immédiatement abandonné.

La justification : Puisque Dijkstra dépile les chemins du moins cher au plus cher, arriver sur v avec un coût plus élevé ET une fatigue plus élevée signifie que ce chemin est strictement dominé. Couper ces branches mortes réduit drastiquement la complexité temporelle.

2.3 Choix 3 : A* et Conception des Heuristiques

Le choix : Remplacer Dijkstra par A* pour guider la recherche, en testant deux heuristiques admissibles :

- **Heuristique Simple (H_1)** : $H_1(v) = \text{DistMin}(v, \text{Arrivée})$. Calculée via un Dijkstra inversé (sans fatigue).
- **Heuristique Dynamique (H_2)** : $H_2(v, F) = \text{DistMin}(v, \text{Arrivée}) \times (1 + F)$.

La justification : Dijkstra explore uniformément autour du départ. A* étire cette recherche vers l'arrivée. H_1 est admissible car le coût réel sera toujours supérieur ou égal à la distance pure. Cependant, sur les réseaux où la fatigue est énorme, H_1 sous-estime trop le coût. Nous avons donc conçu H_2 , qui utilise la fatigue *actuelle* pour proposer une borne plus proche de la réalité.

3 Choix Algorithmiques pour les Missions Multiples

Nous avons adapté notre modèle à un problème d'ordonnancement : un agent doit réaliser k missions (aller de s_i vers t_i).

3.1 Missions avec Ordre Imposé

Le choix : L'état devient $((v, \text{étape}), F)$. La variable "étape" indique l'objectif actuel. Quand la bonne ville est atteinte, l'agent avance d'une étape gratuitement.

Justification : Cela permet d'enchaîner les plus courts chemins en conservant la fatigue accumulée de bout en bout, sans devoir relancer Dijkstra à chaque mission.

3.2 Missions avec Ordre Libre

Le choix : L'état devient $((v, S_{\text{accomplies}}, M_{\text{en_cours}}), F)$. L'ensemble S contient les missions terminées, stockées sous forme de **frozenset**.

Justification : En intégrant l'avancement dans l'état, on transforme un problème d'ordonnancement combinatoire en une recherche de chemin dans un multivers d'états. Le **frozenset** (immuable et non ordonné) est crucial : faire la mission A puis B aboutit au même état logique que faire B puis A, évitant ainsi de recalculer des branches redondantes.

3.2.1 Gestion des collisions dans la file de priorité (L'état "ZZZ_FIN")

Lors du tri des nœuds par la file de priorité (**heapq**), l'algorithme classe les éléments par coût. Sur des instances impliquant plus de 2 missions, le graphe est si dense que de multiples chemins aboutissent au même coût exact. Pour départager cette égalité mathématique parfaite, Python compare la structure de l'état lui-même.

Si l'état final était modélisé par une simple chaîne de caractères (ex : `"etat final"`), Python levait une erreur de type (**TypeError**) en tentant de comparer le tuple d'un état d'exploration normal avec ce texte. Pour résoudre ce problème élégamment, nous avons standardisé la condition d'arrêt sous la forme du tuple suivant : `("ZZZ_FIN", frozenset(), -1)`. L'utilisation de la lettre "Z" garantit une robustesse face au tri alphabétique interne de Python lors de la comparaison des noms de villes, évitant ainsi tout crash de l'algorithme.

4 Présentation et Analyse des Résultats

4.1 Comparaison des algorithmes (Dijkstra vs A*)

Le tableau 1 présente les temps de calcul bruts sur notre graphe implicite.

Fichier Test	Dijkstra	A* (H_1 : Simple)	A* (H_2 : Fatigue)
small	< 0.001 s	< 0.001 s	< 0.001 s
medium-nofatigue	0.0005 s	0.0001 s	0.0001 s
medium-smallfatigue	0.0031 s	0.0033 s	0.0033 s
medium-largefatigue	0.0071 s	0.0073 s	0.0073 s
large-nofatigue	0.0816 s	0.0027 s	0.0021 s
large-smallfatigue	7.7363 s	8.5482 s	8.4741 s
large-largefatigue	<i>Non exécuté</i>	63.3935 s	59.4754 s

TABLE 1 – Temps d'exécution selon l'algorithme et l'heuristique.

Analyse des résultats :

- **Impact de A* (sans fatigue) :** Sur *large-nofatigue*, A* est près de 40 fois plus rapide que Dijkstra (0.0021 s vs 0.0816 s). L'heuristique guide parfaitement l'algorithme vers sa cible.
- **Impact de la fatigue :** Sur les réseaux intermédiaires très fatigants, Dijkstra redevient légèrement plus rapide car le coût de calcul de la fonction heuristique devient supérieur au temps gagné par la réduction de l'espace de recherche.
- **Supériorité de H_2 :** Sur la plus grande instance (*large-largefatigue*), l'heuristique dynamique H_2 prouve sa valeur mathématique en surclassant H_1 de près de 4 secondes (59.4 s contre 63.3 s). Le fait de resserrer la borne en intégrant le multiplicateur de fatigue a permis un élagage bien plus efficace des branches mortes.

4.2 Résultats de l'Extension (Missions Multiples)

Nous avons évalué la complexité de l'ordre libre en testant $k = 2, 4$ et 10 missions sur le graphe *medium-nofatigue*.

Nb. Missions (k)	Coût (Ordre Imposé)	Coût (Ordre Libre)	Temps d'exécution (Libre)
2 missions	3 589	3 589	0.0040 s
4 missions	7 494	5 918	0.0479 s
10 missions	16 033	12 771	17.1648 s

TABLE 2 – Évaluation de l'optimisation par Ordre Libre.

Analyse des résultats (L'explosion combinatoire) : Contrairement aux tests initiaux sur 2 missions (où l'ordre naturel était par hasard le plus rapide), l'augmentation du nombre de missions démontre la puissance de notre modèle : pour 10 missions, l'ordre libre permet de réduire le coût total de plus de 20% (de 16 033 à 12 771).

Cependant, ces résultats illustrent parfaitement la complexité théorique factorielle $\mathcal{O}(k!)$ inhérente à ce problème combinatoire. Pour trouver la combinaison optimale parmi 4 missions ($4! = 24$ ordres possibles), le temps de calcul n'est que de 0.04 seconde. Pour 10 missions (soit plus de 3,6 millions d'ordres possibles à évaluer implicitement), le temps de calcul "explose" pour atteindre 17.16 secondes.

5 Conclusion

Ce projet démontre l'importance de la modélisation en algorithmique. Face à l'explosion combinatoire d'un coût non additif, la solution n'était pas d'optimiser le code Python, mais de repenser l'approche mathématique. La combinaison de l'évaluation paresseuse et de l'élagage de Pareto a permis de résoudre le problème aux limites de la mémoire. Enfin, la modélisation par états complexes a transformé un lourd problème d'ordonnancement de missions en un calcul de plus court chemin robuste, tout en illustrant de manière empirique les limites de calcul posées par les espaces à complexité factorielle.